

# Programmazione a oggetti

---

## Classi e oggetti

*Introduzione alla programmazione orientata agli oggetti in Python.*

## Qual è la funzione di classi e oggetti

Immagina di dover gestire le informazioni di 30 studenti in un programma. Senza un modo per organizzare i dati, ti ritroveresti con decine di variabili sparse ovunque: `nome1` , `eta1` , `voto1` , `nome2` , `eta2` , `voto2` ...

Le **classi** risolvono questo problema: ti permettono di raggruppare dati e funzioni correlate in un unico "pacchetto" chiamato **oggetto**.

---

## La differenza tra classe e oggetto

Una **classe** è come il **progetto di un'automobile**: descrive come deve essere fatta, ma non è un'auto vera.

Un **oggetto** è l'**automobile vera**, costruita seguendo quel progetto.

Dal stesso progetto (classe) puoi costruire molte auto (oggetti) diverse — ognuna con il suo colore, la sua targa, il suo chilometraggio.

---

## Definire una classe

```
class Persona:
    pass    # "pass" significa: classe vuota per ora, aggiungeremo cose dopo
```

## Il costruttore: `__init__`

Il metodo `__init__` è il **costruttore** — viene eseguito automaticamente ogni volta che crei un nuovo oggetto. Serve a impostare i dati iniziali dell'oggetto.

```
class Persona:
    def __init__(self, nome, eta):
        # "self" è l'oggetto che stiamo creando
        # self.nome e self.eta sono i dati dell'oggetto (chiamati
        "attributi")
        self.nome = nome
        self.eta = eta
```

`self` è un riferimento all'oggetto stesso. È sempre il primo parametro di ogni metodo in una classe, anche se quando chiami il metodo non lo scrivi esplicitamente.

## Creare oggetti (istanze)

```
alice = Persona("Alice", 16)  # Crea una Persona con nome="Alice" e eta=16
bob = Persona("Bob", 17)      # Crea una Persona diversa

# Accedere ai dati dell'oggetto con il punto
print(alice.nome)  # Alice
print(bob.eta)     # 17
```

`alice` e `bob` sono due **istanze** diverse della stessa classe `Persona`. Hanno gli stessi "campi", ma valori diversi — come due auto dello stesso modello ma con targhe diverse.

## Aggiungere metodi

I **metodi** sono funzioni definite dentro la classe. Descrivono le **azioni** che un oggetto può fare.

```

class Persona:
    def __init__(self, nome, eta):
        self.nome = nome
        self.eta = eta

    def saluta(self):
        # Questo metodo usa i dati dell'oggetto (self.nome, self.eta)
        print(f"Ciao, mi chiamo {self.nome} e ho {self.eta} anni.")

    def compleanno(self):
        # Questo metodo modifica un dato dell'oggetto
        self.eta += 1
        print(f"Buon compleanno {self.nome}! Ora hai {self.eta} anni.")

alice = Persona("Alice", 16)
alice.saluta()      # → Ciao, mi chiamo Alice e ho 16 anni.
alice.compleanno()  # → Buon compleanno Alice! Ora hai 17 anni.
alice.saluta()      # → Ciao, mi chiamo Alice e ho 17 anni.

```

## Attributi di classe vs attributi di istanza

- **Attributi di istanza:** appartengono al singolo oggetto (ogni oggetto ha il suo valore)
- **Attributi di classe:** sono condivisi da tutti gli oggetti della classe (come una bacheca comune)

```

class Studente:
    scuola = "Liceo Scientifico"  # Attributo di classe: uguale per tutti

    def __init__(self, nome):
        self.nome = nome          # Attributo di istanza: diverso per
ciascuno

alice = Studente("Alice")
bob = Studente("Bob")

print(alice.scuola)  # Liceo Scientifico
print(bob.scuola)    # Liceo Scientifico ← stesso valore per tutti
print(alice.nome)    # Alice
print(bob.nome)      # Bob               ← valore diverso per ciascuno

```

---

## Il metodo `__str__` : come stampare un oggetto

Per default, se fai `print(alice)` Python stampa qualcosa di poco leggibile come `<__main__.Persona object at 0x...>`. Il metodo speciale `__str__` ti permette di decidere tu come appare l'oggetto quando viene stampato.

```
class Persona:
    def __init__(self, nome, eta):
        self.nome = nome
        self.eta = eta

    def __str__(self):
        # Questo viene usato da print() e str()
        return f"Persona(nome={self.nome}, eta={self.eta})"

alice = Persona("Alice", 16)
print(alice)  # → Persona(nome=Alice, eta=16)
```

## Esempio completo: la classe Rettangolo

```
class Rettangolo:
    def __init__(self, base, altezza):
        self.base = base
        self.altezza = altezza

    def area(self):
        # Calcola l'area: base × altezza
        return self.base * self.altezza

    def perimetro(self):
        # Calcola il perimetro: 2 × (base + altezza)
        return 2 * (self.base + self.altezza)

    def __str__(self):
        return f"Rettangolo {self.base}x{self.altezza}"

r = Rettangolo(5, 3)
print(r)           # → Rettangolo 5x3
print(r.area())    # → 15
print(r.perimetro()) # → 16

# Puoi creare più rettangoli indipendenti
r2 = Rettangolo(10, 4)
print(r2.area())   # → 40
```

Ogni rettangolo è un oggetto indipendente: cambiare `r` non influisce su `r2`.

## Ereditarietà in Python

*Come creare classi basate su altre classi in Python.*

## Perché è utile usare l'ereditarietà

Immagina di dover descrivere gli animali in un programma: cani, gatti, uccelli. Tutti respirano, tutti mangiano — ma ognuno fa un suono diverso. Dovresti riscrivere il codice per "respira" e "mangia" tre volte?

L'**ereditarietà** risolve questo problema: scrivi il codice comune una volta sola in una classe "genitore", e le classi "figlie" lo ereditano automaticamente — aggiungendo solo le loro particolarità.

---

## La metafora della famiglia

Pensa all'ereditarietà biologica:

- Un figlio eredita dai genitori molte caratteristiche (colore degli occhi, gruppo sanguigno)
- Ma ha anche caratteristiche sue proprie
- Può anche avere caratteristiche simili ai genitori ma con variazioni (stesso naso, ma di misura diversa)

In programmazione funziona esattamente così.

---

## Sintassi di base

```
class ClassePadre:
    # metodi e attributi del genitore

class ClasseFiglia(ClassePadre):    # ← il nome tra parentesi indica
    "eredita da"
    # eredita tutto dal padre
    # può aggiungere roba nuova o modificare ciò che eredita
```

## Esempio: animali

```
class Animale:
    def __init__(self, nome):
        self.nome = nome

    def respira(self):
        print(f"{self.nome} respira.")

    def mangia(self):
        print(f"{self.nome} mangia.")

# Cane eredita da Animale: ha tutti i suoi metodi + "abbaia"
class Cane(Animale):
    def abbaia(self):
        print(f"{self.nome} fa: Bau bau!")

# Gatto eredita da Animale: ha tutti i suoi metodi + "fa_le_fusa"
class Gatto(Animale):
    def fa_le_fusa(self):
        print(f"{self.nome} fa le fusa...")

fido = Cane("Fido")
fido.respira()      # → Fido respira.    (metodo ereditato da Animale)
fido.mangia()       # → Fido mangia.     (metodo ereditato da Animale)
fido.abbaia()       # → Fido fa: Bau bau! (metodo proprio di Cane)

micio = Gatto("Micio")
micio.respira()     # → Micio respira.
micio.fa_le_fusa()  # → Micio fa le fusa...
```

## Sovrascrivere i metodi (Override)

La classe figlia può **ridefinire** un metodo del genitore per cambiarne il comportamento. È come un figlio che ha "lo stesso tipo di carattere" del padre, ma con sfumature diverse.

```
class Animale:
    def __init__(self, nome):
        self.nome = nome

    def suono(self):
        print("...") # Suono generico

class Cane(Animale):
    def suono(self): # Sovrascrive il metodo del genitore
        print(f"{self.nome} fa: Bau bau!")

class Gatto(Animale):
    def suono(self): # Sovrascrive il metodo del genitore
        print(f"{self.nome} fa: Miao!")

fido = Cane("Fido")
micio = Gatto("Micio")

fido.suono() # → Fido fa: Bau bau!
micio.suono() # → Micio fa: Miao!
```

---

## super() : richiamare il genitore

A volte non vuoi **sostituire** completamente un metodo del genitore, ma vuoi **estenderlo** — aggiungere comportamento in più. `super()` ti permette di chiamare il metodo originale del genitore.



```

class Animale:
    def __init__(self, nome, eta):
        self.nome = nome
        self.eta = eta

    def presenta(self):
        print(f"Mi chiamo {self.nome} e ho {self.eta} anni.")

class Cane(Animale):
    def __init__(self, nome, eta, razza):
        super().__init__(nome, eta)  # Chiama __init__ del genitore per i
        # dati comuni
        self.razza = razza           # Aggiunge il dato extra "razza"

    def presenta(self):
        super().presenta()           # Prima esegui la presentazione del
        # genitore...
        print(f"Sono un {self.razza}.") # ...poi aggiungi info in più

fido = Cane("Fido", 3, "Labrador")
fido.presenta()
# → Mi chiamo Fido e ho 3 anni.
# → Sono un Labrador.

```

# Verificare la parentela tra classi e oggetti

```
class Animale:
    pass

class Cane(Animale):
    pass

fido = Cane()

# isinstance() verifica se un oggetto è di un certo tipo (o di un suo
# "antenato")
print(isinstance(fido, Cane))      # True ← fido è un Cane
print(isinstance(fido, Animale))  # True ← fido è anche un Animale (per
# ereditarietà)

# issubclass() verifica se una classe è "figlia" di un'altra
print(issubclass(Cane, Animale))  # True ← Cane è una sottoclasse di
Animale
```

## Esempio pratico: forme geometriche

```
class Forma:
    def __init__(self, colore):
        self.colore = colore

    def descrivi(self):
        print(f"Sono una forma {self.colore}.")

# Rettangolo eredita da Forma
class Rettangolo(Forma):
    def __init__(self, base, altezza, colore):
        super().__init__(colore)      # Passa il colore al genitore
        self.base = base
        self.altezza = altezza

    def area(self):
        return self.base * self.altezza

# Cerchio eredita da Forma
class Cerchio(Forma):
    def __init__(self, raggio, colore):
        super().__init__(colore)      # Passa il colore al genitore
        self.raggio = raggio

    def area(self):
        return 3.14159 * self.raggio ** 2

r = Rettangolo(5, 3, "rosso")
r.descrivi()      # → Sono una forma rosso.    (metodo ereditato da Forma)
print(r.area())   # → 15

c = Cerchio(4, "blu")
c.descrivi()      # → Sono una forma blu.
print(c.area())   # → 50.265...
```

Entrambe le forme condividono il metodo `descrivi()` — scritto una volta sola nel genitore — e ognuna ha la sua formula per calcolare l'area.

# Iteratori in Python

*Come funzionano gli iteratori in Python.*

## Perché ti serve conoscere gli iteratori

Ogni volta che scrivi `for elemento in lista`, Python sta usando un meccanismo nascosto chiamato **iteratore**. Capire come funziona ti permette di creare le tue sequenze personalizzate — anche sequenze che non finiscono mai, o che generano i dati "al volo" senza occupare memoria.

## La differenza tra iterabile e iteratore

- **Iterabile**: qualcosa su cui puoi fare un ciclo `for`. Liste, stringhe, tuple, dizionari sono iterabili.
- **Iteratore**: un oggetto che "sa dove si trova" nella sequenza e sa come ottenere l'elemento successivo.

Pensa a un libro:

- Il **libro** è l'iterabile (contiene tutti i capitoli)
- Il **segnalibro** è l'iteratore (ricorda a quale pagina sei arrivato)

## Come funziona `for` dentro Python

Quando scrivi:

```
for x in [1, 2, 3]:  
    print(x)
```

Python esegue in realtà questi passi:

1. Chiama `iter([1, 2, 3])` per creare un iteratore dalla lista
2. Chiama ripetutamente `next(iteratore)` per ottenere un elemento alla volta
3. Quando la lista è finita, `next()` lancia `StopIteration` e il ciclo si ferma

Puoi vedere questi passi manualmente:

```
lista = [1, 2, 3]
iteratore = iter(lista)    # Crea il "segnalibro"

print(next(iteratore))     # 1 ← prende il primo elemento e avanza
print(next(iteratore))     # 2 ← prende il secondo elemento e avanza
print(next(iteratore))     # 3 ← prende il terzo elemento e avanza
# print(next(iteratore))   # Qui lancerebbe StopIteration: la lista è
# finita!
```

---

## Creare un iteratore personalizzato

Puoi creare una classe che si comporta come un iteratore implementando due metodi speciali:

- `__iter__` : restituisce l'iteratore stesso
- `__next__` : restituisce il valore successivo (o lancia `StopIteration` se è finita)

```

class ContaDa:
    """Un iteratore che conta da 'inizio' fino a 'fine', incluso."""

    def __init__(self, inizio, fine):
        self.corrente = inizio    # Da dove partiamo
        self.fine = fine         # Dove ci fermiamo

    def __iter__(self):
        return self              # L'iteratore restituisce se stesso

    def __next__(self):
        if self.corrente > self.fine:
            raise StopIteration  # Abbiamo finito!
        valore = self.corrente
        self.corrente += 1       # Avanza al prossimo
        return valore

# Ora possiamo usarlo in un ciclo for, esattamente come una lista
for numero in ContaDa(1, 5):
    print(numero)
# → 1
# → 2
# → 3
# → 4
# → 5

```

## Generatori: il modo più semplice

Creare una classe iteratore completa funziona, ma è verboso. I **generatori** ti permettono di fare la stessa cosa con molto meno codice.

Un generatore è una funzione che usa `yield` invece di `return`. Ogni volta che viene chiamata, si "ferma" a `yield`, restituisce il valore, e riprende da lì alla chiamata successiva — come un libro che segna automaticamente la pagina.

```
def conta_da(inizio, fine):
    corrente = inizio
    while corrente <= fine:
        yield corrente    # "Ecco il valore, mettiti in pausa finché non ti
        serve il prossimo"
        corrente += 1

# Si usa esattamente come prima
for numero in conta_da(1, 5):
    print(numero)
# → 1, 2, 3, 4, 5
```

La differenza rispetto a `return` : con `return` la funzione finisce e dimentica tutto. Con `yield` la funzione si mette in pausa e ricorda esattamente dove si trovava.

## Generator expression: la versione compatta

Come le **list comprehension** (la sintassi `[x**2 for x in range(10)]` che crea una lista in una riga) ma con le parentesi tonde invece di quelle quadre. Producono un generatore invece di una lista: i valori vengono calcolati uno alla volta, solo quando servono, risparmiando memoria.

```
# List comprehension: crea TUTTI i quadrati in memoria subito
quadrati_lista = [x ** 2 for x in range(10)]

# Generator expression: produce UN quadrato alla volta, solo quando serve
quadrati_gen = (x ** 2 for x in range(10))

for q in quadrati_gen:
    print(q)    # → 0, 1, 4, 9, 16, 25, 36, 49, 64, 81
```

## Perché usare i generatori? Il problema della memoria

Immagina di dover lavorare con un milione di numeri:

```
# Senza generatore: tutti i numeri vengono creati in memoria subito
# Con 1.000.000 numeri, questo occupa decine di megabyte di RAM
numeri_lista = list(range(1_000_000))

# Con generatore: i numeri vengono creati uno alla volta, solo quando servono
# Occupa quasi zero memoria – indipendentemente da quanti numeri ci sono
numeri_gen = range(1_000_000)
```

I generatori sono come un nastro trasportatore: producono un pezzo alla volta, invece di mettere tutto sul tavolo in una volta.

## Strumenti pronti: il modulo **itertools**

Python include una raccolta di iteratori già pronti, molto utili:

```
# Ciclo infinito su una sequenza (utile ad esempio per semafori o animazioni)
colori = itertools.cycle(["rosso", "giallo", "verde"])
for i in range(6):
    print(next(colori))    # → rosso, giallo, verde, rosso, giallo, verde

# Unire più sequenze in una sola
for x in itertools.chain([1, 2], [3, 4], [5]):
    print(x)    # → 1, 2, 3, 4, 5

# Prendere solo i primi n elementi di una sequenza (anche infinita)
for x in itertools.islice(range(1_000_000), 5):
    print(x)    # → 0, 1, 2, 3, 4
```